Task 0

Refer to `lab-prep.txt` for plaintext output. Setup result for python3 and openssl environment

```
None
$ python3 --version
Python 3.12.3

$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)

$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and
primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors
<cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laugher, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger
Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
```

Task 1

Refer to `task1.txt` for plaintext output. Output from running `baseline_ctr.py`.

```
None
======================================================================
Part A: CTR bit-flip (malleability)
======================================================================
original plaintext : b'{"action":"READ","path":"notes.txt"}' (36 bytes)
IV (public)        : f4ba73feaab8bb874f7322b33f2fa7a5
original ciphertext:
e9913c72a8fea83579740737247dbaf362ba9b29c81f8ae5feb4257e4e5a898ce2515a16
relay changes b'READ' -> b'EXEC' at byte offset 11 (no key used)
delta = old XOR new: 171d0407
forged ciphertext  :
e9913c72a8fea835797407203979bdf362ba9b29c81f8ae5feb4257e4e5a898ce2515a16
XOR relation  C XOR C'  ==  P XOR P'
  C  XOR C' :
000000000000000000000171d0407000000000000000000000000000000000000000000
  P  XOR P' :
000000000000000000000171d0407000000000000000000000000000000000000000000
  equal?      True
Receiver gets the forged ciphertext:
  [receiver] processed message #1: b'{"action":"EXEC","path":"notes.txt"}'
  -> accepted with no error: the attacker changed the command.
======================================================================
Part B: replay
======================================================================
Sender sends one legitimate message; attacker records and resends it.
  [receiver] processed message #1: b'{"action":"READ","path":"notes.txt"}'
  [receiver] processed message #2: b'{"action":"READ","path":"notes.txt"}'
messages processed from ONE send: 2
  -> receiver cannot tell a replay from the original.
```

My implementation for `baseline_ctr.py` emulates the sending of an encrypted message using python functions that operate as the sender and receiver. A relay function serves as a man-in-the-middle, intercepting the traffic from the sender and relaying it to the receiver.

The relay is able to forge the ciphertext and replace the READ action with EXEC and send multiple copies of a legitimate message to the receiver. CTR allows confidentiality in communication but lacks integrity due to the malleability of the ciphertext.

Flipping a bit in the ciphertext results in a corresponding bit in the plaintext to flip as well. Note that the relay never learns what the key actually is but does know the old bytes (READ) and

their offset in the message. By exploiting the XOR relationship of $C \oplus C' = P \oplus P'$ between the original and modified ciphertext and plaintext, the relay can modify a specific part of the ciphertext to forge the EXEC action into the payload.

The protocol itself also has no freshness check which allows the relay to replay multiple copies of the message to the receiver. The receiver is stateless and will process each copy of the message as legitimate messages.

Task 2

Refer to `task2.txt` for plaintext output. Output from running `handshake.py`.

```
None
loaded ffdhe3072 group (3072-bit p, g = 2)
generating 3072-bit RSA keys (slow under emulation)...
Valid handshake
  session_id (gateway): 40691344bf3d875a
  session_id (node)   : 40691344bf3d875a
  transcript hashes match: True
  four directional keys all distinct: True
Attacks
  [rejected] changed nonce: invalid RSA-PSS signature
  [rejected] unexpected identity: unexpected peer identity
  [rejected] reflected hello: unexpected peer identity
  [rejected] malformed transcript: malformed transcript: declared length exceeds
data
```

My implementation for `handshake.py` emulates an in-process handshake between a gateway and node. Both sides send hello messages containing its identity, Diffie-Hellman public value, and 16-byte nonce while signing the handshake as the message is passed between the two entities. Both sides also maintain a transcript containing information about the handshake such as protocol name, group identifier (ffdhe3072 in this case), other identifying information, and the public DH values and nonces of the two sides. A hash of this transcript is then concatenated with the role to sign the handshake.

Both sides generate a private and public value using the group ffdhe3072. Combining its own private value with the other public value generates the same secret Z without having to send it through the handshake. As the DH values are discarded after the session ends (ephemeral), compromising the RSA key does not reveal any past session keys (forward secrecy).

DH does not provide authentication in this case. Instead, the RSA signature verifies the peer's signature against a public key to ensure that the sender of the message is expected. Any unexpected entity is rejected. DH on the other hand is used to generate the shared secret the two sides will use on further messaging.

As mentioned above, the transcript holds the nonce values for both the gateway and the node, making the signature value for only this particular run. Replaying a hello from a prior session from either the gateway or the node fails. Replaying an old ServerHello the nonce and DH value would be different. Replaying an old ClientHello cannot reproduce the gateway's matching signature as its nonce is fresh.

As the handshake is signed using its own roles, nonces, and DH values, a man-in-the-middle attack that tampers with the handshake by inserting their own DH values also changes the transcript, failing the signature. A mismatched identity, nonce, or DH value results in a rejection. This also guarantees that a signature from one side cannot be misinterpreted or echoed back as the other's. Any mismatch in the transcript ends up generating different keys.

In the case where the signatures were removed, this effectively removes any authentication in the handshake. An attacker could run separate DH handshakes between the gateway and node while tampering with the traffic and neither would be able to tell. As stated before, RSA is needed to enforce authentication. Without it, simple DH cannot enforce identification.

Task 3

Refer to `task3.txt` for plaintext output. Output from running `secure_record.py`.

```
None
record: 83 bytes = 15 header + 36 ct + 32 tag
accepted: type=1 plaintext=b'{"action":"READ","path":"notes.txt"}'
rejected replay: ReplayedRecord
rejected bit-flipped ciphertext: AuthenticationFailure
```

Task 4

Refer to `task4.txt` for plaintext output. Output from running the test cases inside `/tests`. Refer to the `README.md` file for instructions on how to run the test cases (note that the python3 and openssl setup may be required to ensure the proper python packages and group is present).

None

```
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 --
/home/ubuntu/csce465-agentsec/hw2/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/ubuntu/csce465-agentsec/hw2
collecting ... collected 71 items
tests/test_handshake_attacks.py::test_incorrect_rsa_public_key_is_rejected PASSED
[  1%]
tests/test_handshake_attacks.py::test_corrupted_node_signature_is_rejected PASSED
[  2%]
tests/test_handshake_attacks.py::test_corrupted_gateway_signature_is_rejected
PASSED [  4%]
tests/test_handshake_attacks.py::test_well_formed_signature_from_an_impostor_key_i
s_rejected PASSED [  5%]
tests/test_handshake_attacks.py::test_signature_over_the_wrong_role_is_rejected
PASSED [  7%]
tests/test_handshake_attacks.py::test_client_hello_reflected_back_to_gateway
PASSED [  8%]
tests/test_handshake_attacks.py::test_server_hello_reflected_back_to_node PASSED [
9%]
tests/test_handshake_attacks.py::test_peer_echoing_our_nonce_is_rejected PASSED [
11%]
tests/test_handshake_attacks.py::test_peer_echoing_our_dh_value_is_rejected PASSED
[ 12%]
tests/test_handshake_attacks.py::test_node_signature_reflected_as_gateway_finished
PASSED [ 14%]
tests/test_handshake_attacks.py::test_replayed_client_hello_is_out_of_state PASSED
[ 15%]
tests/test_handshake_attacks.py::test_server_hello_before_client_hello_is_out_of_s
tate PASSED [ 16%]
tests/test_handshake_attacks.py::test_changed_node_nonce_is_rejected PASSED [ 18%]
tests/test_handshake_attacks.py::test_changed_client_nonce_breaks_the_signature
PASSED [ 19%]
tests/test_handshake_attacks.py::test_changed_dh_public_value_is_rejected PASSED [
21%]
```

```
tests/test_handshake_attacks.py::test_man_in_the_middle_dh_substitution_is_rejecte
d PASSED [ 22%]
tests/test_handshake_attacks.py::test_unexpected_identity_is_rejected_by_both_side
s PASSED [ 23%]
tests/test_handshake_attacks.py::test_degenerate_dh_public_values_are_rejected[zer
o-0] PASSED [ 25%]
tests/test_handshake_attacks.py::test_degenerate_dh_public_values_are_rejected[one
-1] PASSED [ 26%]
tests/test_handshake_attacks.py::test_degenerate_dh_public_values_are_rejected[p-1
-None] PASSED [ 28%]
tests/test_handshake_attacks.py::test_dh_public_value_of_wrong_length_is_rejected
PASSED [ 29%]
tests/test_handshake_attacks.py::test_nonce_of_wrong_length_is_rejected PASSED [
30%]
tests/test_handshake_attacks.py::test_a_valid_transcript_parses PASSED   [ 32%]
tests/test_handshake_attacks.py::test_malformed_transcript_is_rejected_before_hash
ing[declared_length_too_large] PASSED [ 33%]
tests/test_handshake_attacks.py::test_malformed_transcript_is_rejected_before_hash
ing[truncated_length_prefix] PASSED [ 35%]
tests/test_handshake_attacks.py::test_malformed_transcript_is_rejected_before_hash
ing[truncated_last_field] PASSED [ 36%]
tests/test_handshake_attacks.py::test_malformed_transcript_is_rejected_before_hash
ing[trailing_extra_field] PASSED [ 38%]
tests/test_handshake_attacks.py::test_malformed_transcript_is_rejected_before_hash
ing[missing_field] PASSED [ 39%]
tests/test_handshake_attacks.py::test_malformed_transcript_is_rejected_before_hash
ing[wrong_label] PASSED [ 40%]
tests/test_handshake_attacks.py::test_malformed_transcript_is_rejected_before_hash
ing[wrong_group] PASSED [ 42%]
tests/test_handshake_attacks.py::test_malformed_transcript_is_rejected_before_hash
ing[short_dh_value] PASSED [ 43%]
tests/test_handshake_attacks.py::test_malformed_transcript_is_rejected_before_hash
ing[short_nonce] PASSED [ 45%]
tests/test_handshake_attacks.py::test_length_prefixes_make_the_encoding_unambiguou
s PASSED [ 46%]
tests/test_modified_ciphertext.py::test_flipped_ciphertext_bit_is_rejected[first]
PASSED [ 47%]
tests/test_modified_ciphertext.py::test_flipped_ciphertext_bit_is_rejected[middle]
PASSED [ 49%]
tests/test_modified_ciphertext.py::test_flipped_ciphertext_bit_is_rejected[last]
PASSED [ 50%]
tests/test_modified_ciphertext.py::test_task1_bitflip_attack_is_now_detected
PASSED [ 52%]
tests/test_modified_ciphertext.py::test_modified_tag_is_rejected PASSED  [ 53%]
```

```
tests/test_modified_ciphertext.py::test_truncated_ciphertext_is_malformed PASSED [
54%]
tests/test_modified_ciphertext.py::test_extended_ciphertext_is_malformed PASSED [
56%]
tests/test_modified_ciphertext.py::test_truncating_and_fixing_the_length_field_sti
ll_fails_the_mac PASSED [ 57%]
tests/test_modified_ciphertext.py::test_record_shorter_than_header_plus_tag_is_mal
formed PASSED [ 59%]
tests/test_modified_header.py::test_modified_header_field_is_rejected[version]
PASSED [ 60%]
tests/test_modified_header.py::test_modified_header_field_is_rejected[direction]
PASSED [ 61%]
tests/test_modified_header.py::test_modified_header_field_is_rejected[sequence-hig
h] PASSED [ 63%]
tests/test_modified_header.py::test_modified_header_field_is_rejected[sequence-low
] PASSED [ 64%]
tests/test_modified_header.py::test_modified_header_field_is_rejected[message-type
] PASSED [ 66%]
tests/test_modified_header.py::test_modified_header_field_is_rejected[length]
PASSED [ 67%]
tests/test_modified_header.py::test_attacker_cannot_rewrite_sequence_to_dodge_repl
ay_detection PASSED [ 69%]
tests/test_modified_header.py::test_header_is_readable_but_authenticated PASSED [
70%]
tests/test_record_reflection.py::test_gateway_record_reflected_back_to_gateway
PASSED [ 71%]
tests/test_record_reflection.py::test_node_record_reflected_back_to_node PASSED [
73%]
tests/test_record_reflection.py::test_rewriting_the_direction_byte_does_not_help
PASSED [ 74%]
tests/test_record_reflection.py::test_directions_use_different_keys PASSED [ 76%]
tests/test_record_reflection.py::test_reflection_does_not_release_plaintext PASSED
[ 77%]
tests/test_replay.py::test_immediate_replay_is_rejected PASSED           [ 78%]
tests/test_replay.py::test_replay_of_an_old_record_after_later_traffic PASSED [
80%]
tests/test_replay.py::test_dropped_record_is_detected_and_state_survives PASSED [
81%]
tests/test_replay.py::test_reordered_delivery_is_rejected PASSED         [ 83%]
tests/test_replay.py::test_record_from_another_session_is_rejected PASSED [ 84%]
tests/test_replay.py::test_sender_never_reuses_a_sequence_number_or_iv PASSED [
85%]
tests/test_replay.py::test_sender_refuses_to_wrap_the_sequence_number PASSED [
87%]
```

```
tests/test_valid_handshake.py::test_both_sides_derive_identical_session PASSED [
88%]
tests/test_valid_handshake.py::test_all_four_directional_keys_are_distinct PASSED
[ 90%]
tests/test_valid_handshake.py::test_messages_flow_in_both_directions PASSED [ 91%]
tests/test_valid_handshake.py::test_sequence_numbers_start_at_zero_in_each_directi
on PASSED [ 92%]
tests/test_valid_handshake.py::test_directions_keep_independent_counters PASSED [
94%]
tests/test_valid_handshake.py::test_record_size_is_header_plus_ciphertext_plus_tag
PASSED [ 95%]
tests/test_valid_handshake.py::test_ciphertext_does_not_contain_the_plaintext
PASSED [ 97%]
tests/test_valid_handshake.py::test_empty_and_large_payloads_round_trip PASSED [
98%]
tests/test_valid_handshake.py::test_each_handshake_produces_a_new_session PASSED
[100%]
============================== 71 passed in 3.81s ==============================
```

The attacker in this scenario is an active on-path foe who can read, modify, drop, replay, reorder, reflect, and inject messages sent between two entities (node and gateway in the case for task 2). However, the attacker does not have any keys. The attacker is NOT trying to compromise endpoints, analyze traffic, deny service, or decide whether an authorized tool call is safe.

The CTR mode used here requires a unique IV. Note that “unique” in this case refers to per key rather than per session. Session keys are fresh on each handshake, sequence numbers are never reused, separate keys are used per direction, and so the same IV in a different session does not compromise the protocol. In the case where the same IV is used with the same key, the keystream generated is exactly the same. Reusing the IV ends up making the cipher behave like a two-time pad. Using the relation $C1 \oplus C2 = P1 \oplus P2$ and with a guessable plaintext (such as JSON format used in task 1), the attacker can recover the message with the use of a key. The IV is also the starting counter and any records over 16 bytes share keystream blocks with their neighbor even though no IV repeats. Therefore, the confidentiality of overlapping blocks is weakened. An attacker can see that two ciphertexts are encrypted with the same keystream and can make an educated guess on how to decrypt the corresponding blocks. This is further compounded by the fact that the plaintext uses a relatively fixed format (JSON with fixed command size from task 1).

Verify-before-decrypt matters as you should never decrypt a potentially tampered message until you verify that it is authentic. This avoids having to worry about discarding any tampered plaintext, shrinks the attack surface a malicious attacker could use, and rejects any bad records

cheaply without having to spend time on decrypting a message that would be discarded anyway. The verification itself is also executed in constant time which prevents attackers from using timing based attacks.

Having independent encryption and MAC keys ensures that a flaw in one does not impact the other. The `secure_record.py` derives the keys with different labels and rejects equal keys outright. The proof that encrypt-then-MAC is secure assumes independent keys each used for one purpose.

GCM does both encryption and authentication in one primitive using one key. Its nonce rules are structured as reuse leaks plaintext relationships that can also break authentication. This construction has two passes and two separate keys and relies on the composition of the code and protocol being correct. AEAD is harder to misuse than the encrypt-then-MAC based protocol used here.

The MAC proves who sent the record and that it wasn't tampered with. It says nothing about the meaning of the message itself. Any authenticated gateway or compromised or manipulated agent can send a valid message holding an action to execute some malicious or unintended action. As a matter of fact, a human may make the same mistake while still proving that you are you and that nobody else tampered with your instructions.